Diffusion Models

Positioning GANs in the Course

1	2	3	4
Classical Methods	Autoencoders	GANs	Diffusion Models

Opening questions

- Why did diffusion models overtake GANs as the dominant paradigm?
- How can destroying data with noise help us generate new data?
- Why are diffusion models more stable to train than GANs?
- What made text-to-image generation suddenly possible at scale?

Three Paradigms Revisited

VAE

- Explicit probabilistic model
- Encoder-decoder with latent z
- Stable training (ELBO)
- Often blurrier outputs
- Fast single-pass generation

GAN

- Implicit adversarial model
- Generator vs discriminator
- Unstable (mode collapse)
- Sharp, realistic samples
- Fast single-pass generation

Diffusion

- Iterative denoising model
- Forward noise + learned reverse
- Very stable training
- High quality + diversity
- Slow multi-step generation

Diffusion models combine the stability of VAEs with quality that rivals or exceeds GANs

01

The Core Idea

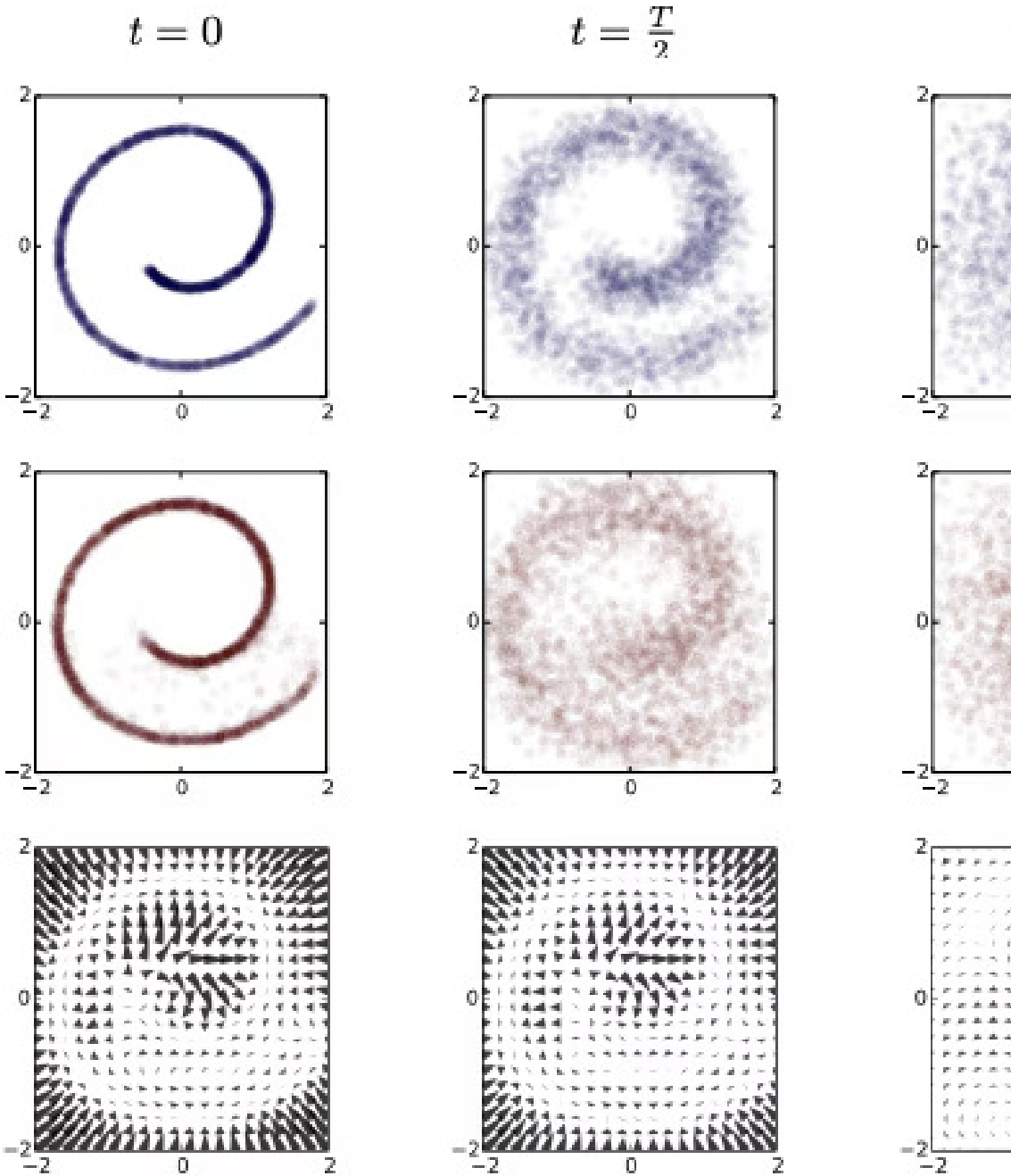
From noise destruction to learned restoration

The Foundational Paper

Original paper: Sohl-Dickstein et al. — "Deep Unsupervised Learning using Nonequilibrium Thermodynamics", ICML 2015

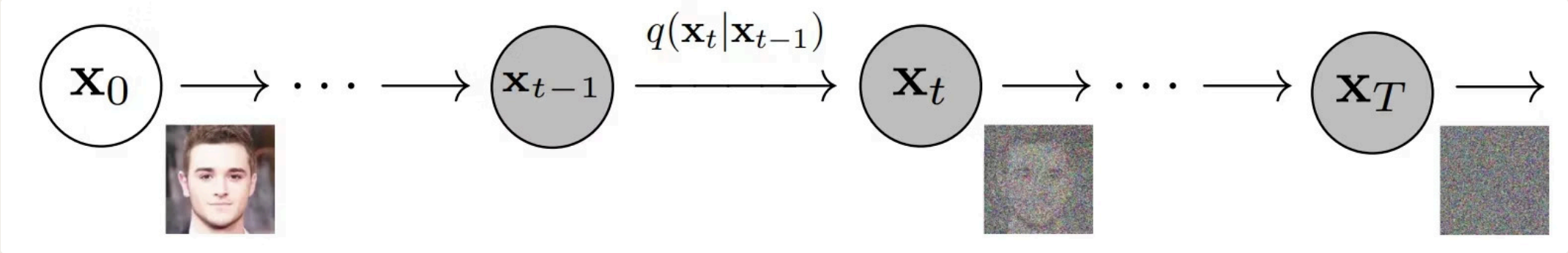
> **Key contributions**
>
> - Inspired by nonequilibrium statistical mechanics
> - First forward/reverse diffusion framework
> - A generative model defined by reversing a simple corruption process — no adversarial training, no complex inference

The paper that started it all — even if the field took 5 years to notice

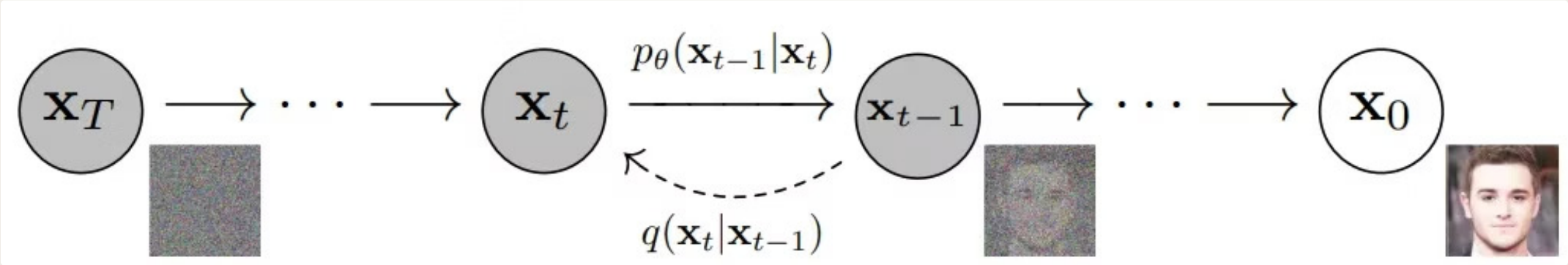


The Foundational Paper

Forward process: Add noise



Reverse process: remove noise (using a NN model)



Blackboard Time

The Key Insight

Destroying data is easy. Learning to reverse the destruction is the generative model.

Forward Process (Fixed)

Gradually add Gaussian noise

Each step adds a small amount of noise

After T steps: pure noise

No learning required — just a schedule

Reverse Process (Learned)

Gradually remove noise

A neural network predicts the noise

After T steps: clean image

This is the generative model

Denoising Diffusion Probabilistic Models (DDPM)

Key paper: Ho et al. — "Denoising Diffusion Probabilistic Models" (DDPM), NeurIPS 2020

The algorithms

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

02

The Forward Process

Gradually destroying data with Gaussian noise

Forward Process: Adding Noise

Single Step

$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right)$

At each step t, scale down the signal and add Gaussian noise with variance β_t

Jump to any Timestep (Reparameterization)

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \alpha_1 \alpha_2 \cdots \alpha_t$

$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I)$

This means: $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$

We can jump directly to any noise level t without iterating — crucial for efficient training

The Noise Schedule

β_t controls how much noise is added at each step. The schedule defines the destruction trajectory.

Linear Schedule (DDPM)

β grows linearly from β_t = 10^{-4} to β_t = 0.02

T = 1000 steps

Simple but not optimal — wastes steps at low noise levels

Cosine Schedule

$\bar{a}_t$ follows a cosine curve

Smoother transition from clean to noisy

Better for lower resolutions

Improved DDPM (Nichol & Dhariwal, 2021)

What the Schedule Means

$t = 0 : \bar{a}_t \approx 1 \rightarrow x_t \approx x_0$ (almost clean)

$t = T/2 : \ \bar{a}_t \approx 0.5 \rightarrow x_t$ (is half signal, half noise)

$t = T : \ \bar{a}_t \approx 0 \rightarrow x_t \approx \varepsilon$ (pure noise)

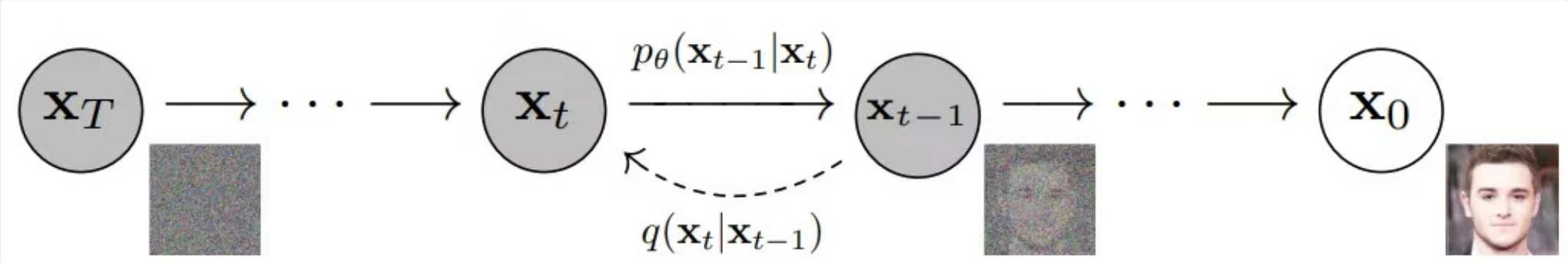
03

The Reverse Process

Learning to denoise, one step at a time

Reverse Process: From Noise to Image

Generation is iterative refinement — the model denoises one step at a time



> ***Unlike GANs (one-shot generation), diffusion models refine the output gradually — each step removes a small amount of noise***

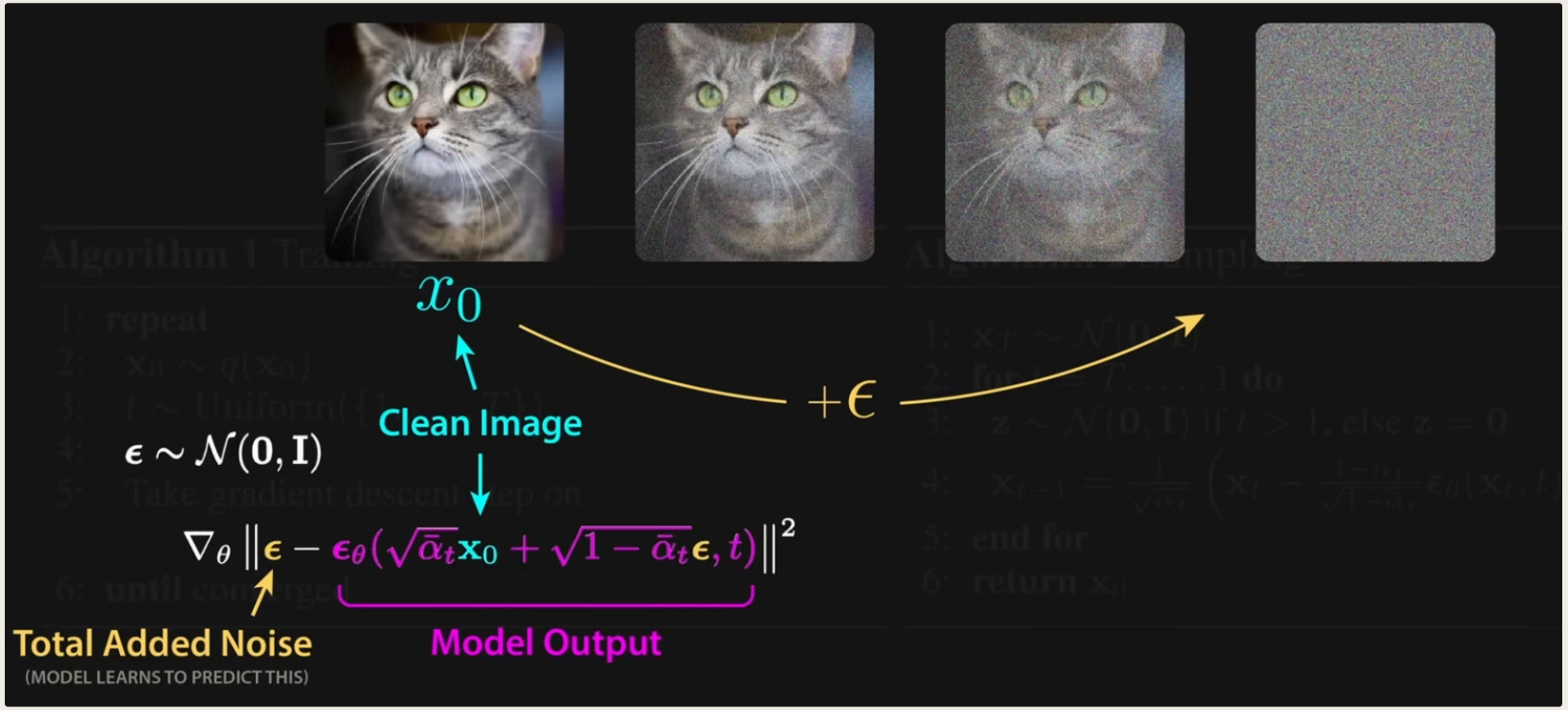
A neural network p_{theta} learns to predict and remove the noise at each step. This is the part we train.

Denoising Diffusion Probabilistic Models (DDPM)

Key paper: Ho et al. — "Denoising Diffusion Probabilistic Models" (DDPM), NeurIPS 2020

Trick 1: Predict the total noise added to the image, not step by step!!

It appears more difficult, but looks can be deceiving!!



Blackboard Time

Reverse Process: Learning to Denoise

What We Want

$$p_\theta(x_{t-1}|x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

A neural network that, given noisy x_t and timestep t, predicts the slightly less noisy x_{t-1}

Conditioned by time!!!

DDPM Simplification

Fix variance: $\Sigma = \beta_t * I$ (not learned)

Instead of predicting μ directly, predict the noise ε that was added

The network $\varepsilon_\theta(x_t, t)$ outputs the noise estimate

Why Predict Noise?

Predicting noise is equivalent to predicting μ

But empirically it works much better

Connects to score matching theory

What Does the Network Predict?

The model receives a noisy image x_t and the timestep t. But what is the training target?

Input

Noisy image x_t

Timestep t (how much noise)

The network must figure out what was noise and what was signal

Output (DDPM)

The predicted noise ε

From the predicted noise, we can estimate the clean image x_0

This is especially stable and convenient

Alternative Parameterizations

Predict epsilon (noise) — the DDPM approach, most common and stable

Predict x_0 (clean image) — direct reconstruction, used in some formulations

Predict v (velocity) — a blend of x_0 and epsilon, better for certain schedules

Key insight: the model does not generate an image in one shot — it learns to estimate the noise component at each timestep

Training Objective

$$L = \mathbb{E}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$$

Simple MSE between the actual noise and the predicted noise

> ### Training Algorithm
>
> 1. Sample a clean image x_0 from the dataset
> 2. Sample a random timestep $t \sim \mathrm{Uniform}(1, T)$
> 3. Sample noise ε $N(0, I)$
> 4. Construct noisy image: $x_t = sqrt(\bar{a}_t) * x_0 + sqrt(1 - \bar{a}_t) * \varepsilon$
> 5. Predict noise: $\hat{\epsilon} = \epsilon_\theta(x_t, t)$
> 6. Compute loss: $L = \|\epsilon - \hat{\epsilon}\|^2$ and backpropagate
>
> ***No adversarial game, no mode collapse, no training instability — just regression***

Why This Works

Variational Bound

DDPM loss is a simplified version of a variational lower bound (ELBO)

Similar spirit to VAEs: maximize a lower bound on log-likelihood

But the simplified MSE loss works as well or better in practice

Score Matching View

Predicting noise ε is equivalent to estimating the score:

$\nabla_x \log p(x_t)$

The gradient of the log-density tells us which direction leads to higher-probability data

The Unifying Perspective

Song et al. (2021) showed that DDPM and score-based models are two views of the same framework:

Both model a continuous-time SDE that transforms data to noise

The reverse SDE generates samples by following the learned score

The Denoising Network: U-Net

DDPM uses a U-Net with timestep conditioning — not just any network

Key Components

- ResNet blocks with GroupNorm
- Self-attention at lower resolutions
- Skip connections (encoder → decoder)
- Sinusoidal timestep embeddings

Timestep injected via FiLM conditioning

Why U-Net?

- Skip connections preserve spatial detail
- Multi-scale processing for different noise levels
- Same architecture works for all timesteps
- Proven in segmentation, adapted for generation

Alternative: Transformer-based (DiT)

04

Sampling

Generating images from pure noise

Denoising Diffusion Probabilistic Models (DDPM)

Key paper: Ho et al. — "Denoising Diffusion Probabilistic Models" (DDPM), NeurIPS 2020

Trick 2: Noise added during training and during generation!!!

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$$
5: **end for**
6: return $\mathbf{x}_0$

Output Image

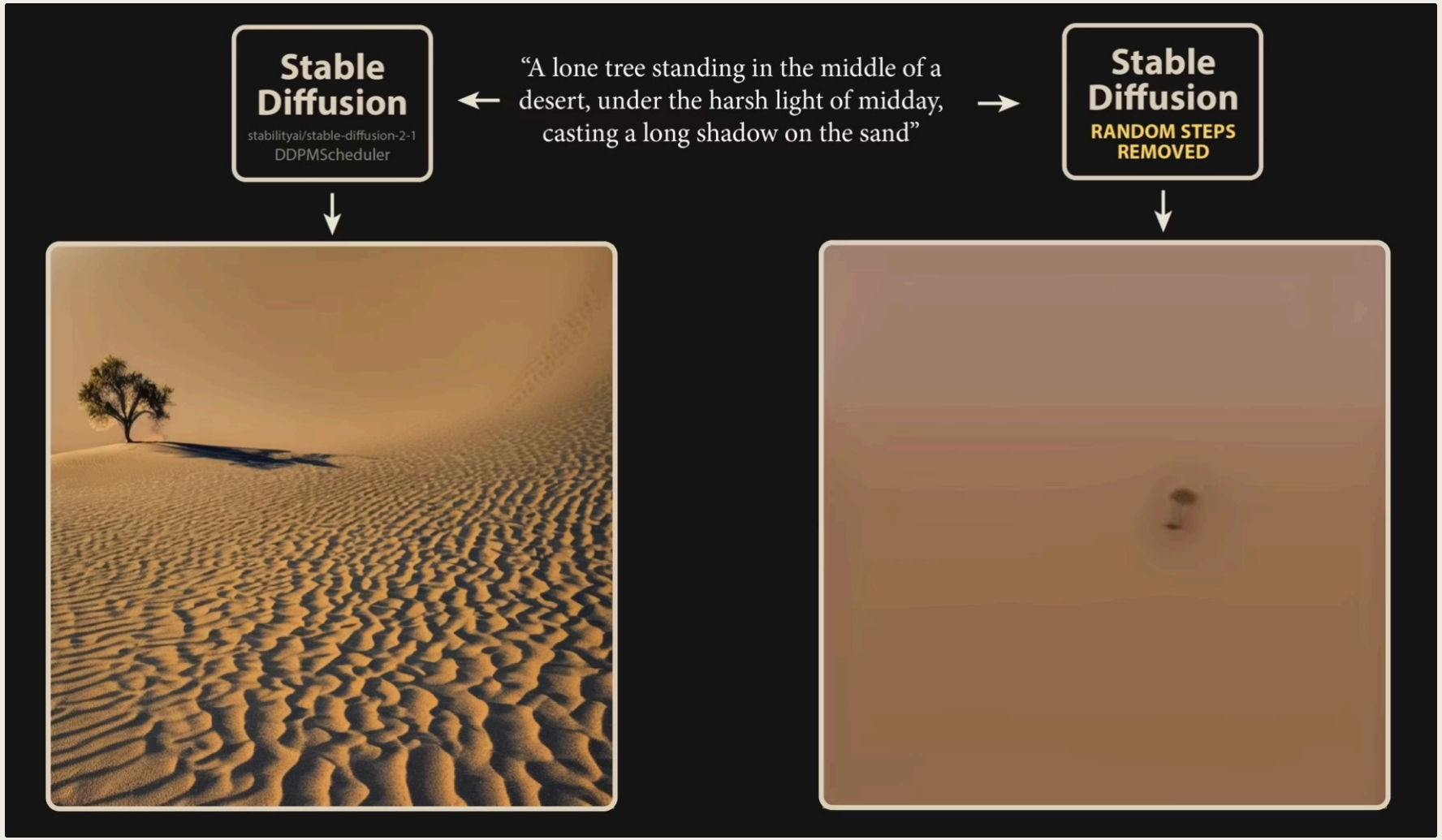
Scaled Model Output

Random Noise Image

Denoising Diffusion Probabilistic Models (DDPM)

Key paper: Ho et al. — "Denoising Diffusion Probabilistic Models" (DDPM), NeurIPS 2020

Trick 2: Noise added during training and during generation!!!



Blackboard Time

DDPM Sampling Algorithm

1. $x_T \sim \mathcal{N}(0, I) \qquad \#\ \text{start from pure noise}$
2. $\text{for } t = T, T-1, \ldots, 1:$

$$z \sim \mathcal{N}(0, I) \text{ if } t > 1, \text{ else } z = 0$$

$$\hat{\epsilon} = \epsilon_\theta(x_t, t) \qquad \#\ \text{predict the noise}$$

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\epsilon} \right) + \sqrt{\beta_t}\, z$$

T = 1000 steps

Very slow! Each step requires a full U-Net forward pass

Stochastic sampling

The z term adds fresh noise — gives diversity

DDIM: Faster Sampling

Key Paper: Song et al. (2020) — Denoising Diffusion Implicit Models

Key Insight

DDPM's Markov chain is not the only way to reverse the forward process

DDIM defines a non-Markovian process that shares the same training objective

The math is complicated!

Same trained model, different sampler

Benefits

Deterministic sampling (no z noise)

Skip steps: use 50-100 instead of 1000

10-50x speedup at sampling time

Consistent outputs from same z (interpolation-friendly)

DDPM: 1000 steps, stochastic, diverse | DDIM: 50 steps, deterministic, fast

Other fast samplers: DPM-Solver, Euler, Heun, PNDM — an active research area

05

Guidance

Steering generation with conditions and labels

05.1

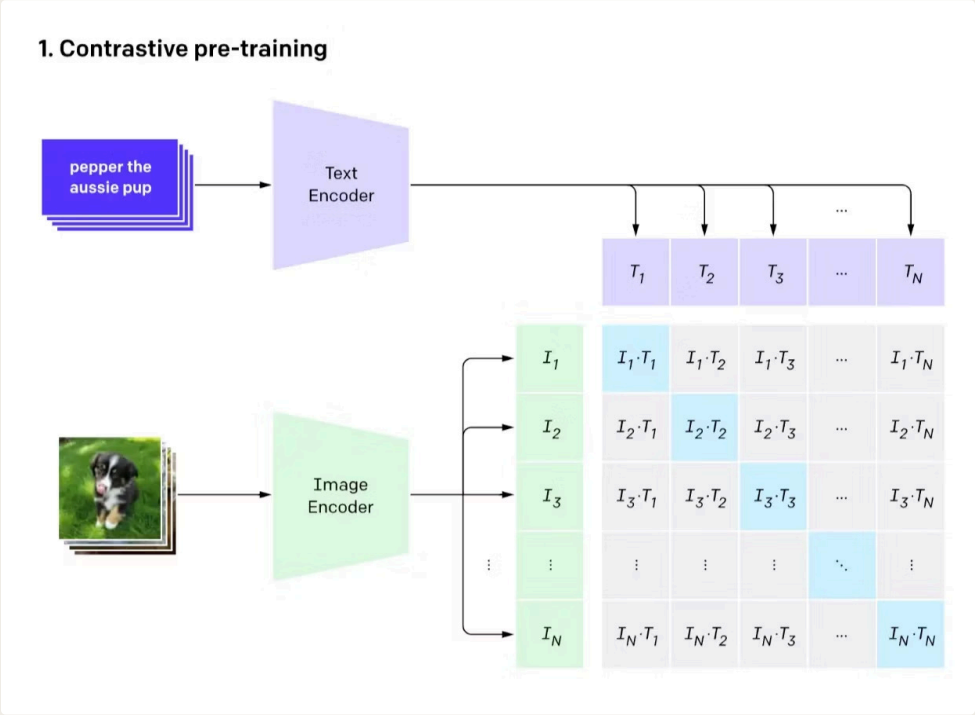
CLIP

Contrastive Language–Image Pre-training

Contrastive Language–Image Pre-training

Key paper: Radford et al. — Learning Transferable Visual Models From Natural Language Supervision, ICML 2021

- Contrastive Pretraining: It is trained to distinguish matching from non-matching pairs to create versatile representations for various tasks.
- Zero-Shot Capabilities: can classify new images based on textual descriptions
- Broadly used in combination with image generative approaches. In most works, provided a way to match between the prompts text features and image features.



The Intuition Behind Guidance

How to steer diffusion?

Blackboard Time

The Intuition Behind Guidance

How do we steer a diffusion model to generate what we want?

Unconditional

"Generate any plausible image"

- Model generates freely
- No prompt or label
- High diversity
- No control over content

Conditional

"Generate a cat"

- Model receives a condition c (text, class label, etc.)
- Outputs are relevant
- But sometimes weakly aligned

Guided

"Definitely a cat!"

- Compare conditional vs unconditional predictions
- Push harder toward the prompt
- Guidance scale controls strength

Guidance = amplify the difference between "what the model would generate anyway" and "what the prompt asks for"

Higher guidance: better prompt adherence, but less diversity. Too high: artifacts and over-saturation.

Conditional Generation

How do we go from "generate any image" to "generate an image of a cat"?

Conditioning Inputs

- Class label (e.g., ImageNet class)
- Text prompt (via CLIP / T5 embeddings)
- Image (inpainting, super-resolution)
- Spatial map (segmentation, depth, pose)
- **The noise predictor becomes:** $\varepsilon_\theta(x_t, t, c)$

Classifier Guidance

Use a separate pretrained classifier to steer sampling toward the desired class

Classifier-Free Guidance (CFG)

No extra classifier needed — the model learns both conditional and unconditional denoising

CFG is the SOTA

Classifier Guidance

Dhariwal & Nichol (2021) — "Diffusion Models Beat GANs on Image Synthesis"

The formula

Train a classifier $p(y|x_t)$ on noisy images

At sampling time, shift the noise prediction using the classifier gradient:

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t) - s\sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t} \log p(y \mid x_t)$$

where s is the guidance scale (higher = stronger conditioning)

Advantages

First method to beat GANs on FID

Can use any pretrained classifier

Tunable quality-diversity tradeoff via s

Limitations

Requires a separate noisy classifier

Classifier must handle all noise levels

Doesn't scale well to text conditioning

Classifier-Free Guidance

Ho & Salimans (2022) — the method that powers modern text-to-image models

The Trick

During training, randomly drop the condition *c* with some probability (e.g., 10%)

The model learns both $\varepsilon_\theta(x_t, t, c)$ and $\varepsilon_\theta(x_t, t, \emptyset)$

At sampling, extrapolate away from unconditional toward conditional:

$$\epsilon_{\text{cfg}} = \epsilon_\theta(x_t, t, \varnothing) + w\,(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing))$$

The Guidance Scale w

w = 1.0: standard conditional generation (no guidance)

w = 3-8: sweet spot — high quality and good text alignment

w > 15: over-saturated, artifacts, reduced diversity

CFG trades diversity for quality and prompt faithfulness

06

Latent Diffusion

Moving diffusion from pixel space to latent space

The Resolution Problem

Key paper: *Rombach et al. (2022) — "High-Resolution Image Synthesis with Latent Diffusion Models" (LDM)*

Pixel-Space Diffusion

U-Net operates on full-resolution images

256x256x3 = 196,608 dimensions

512x512 = 786,432 dimensions

Extremely expensive: memory and compute scale quadratically with resolution

DDPM trained on 256x256 already costly

The Solution: Latent Space

Compress images to a latent representation first

64x64x4 = 16,384 dimensions

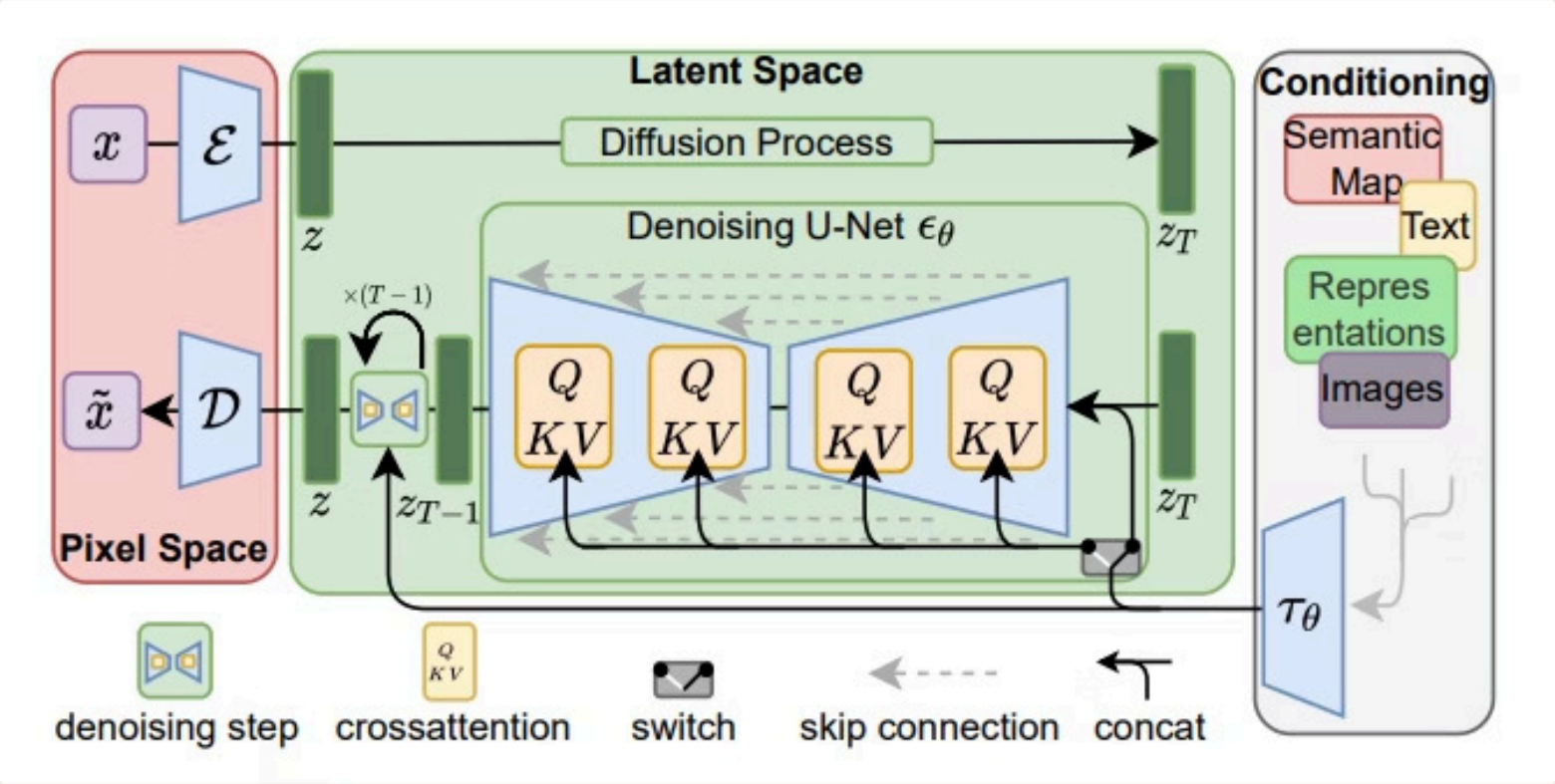
That's 48x fewer dimensions!

Run diffusion in this compact space

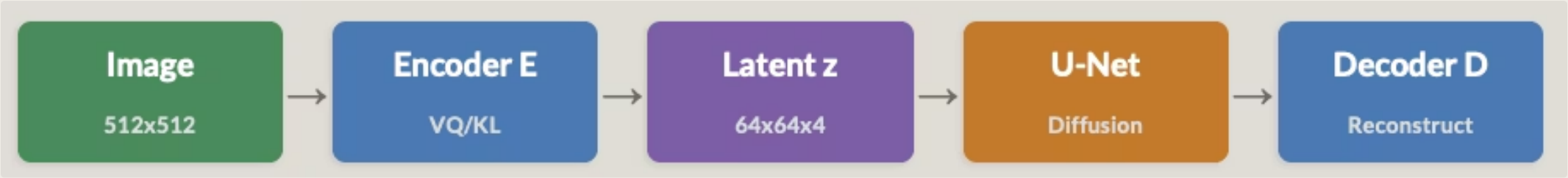
Decode back to pixel space at the end

Latent Diffusion = pretrained autoencoder + diffusion in latent space

Latent Diffusion Architecture



Latent Diffusion Architecture



Stage 1: Autoencoder

- Trained separately (VQ-VAE or KL-VAE)
- Compresses 512x512 to 64x64x4
- Preserves perceptual quality
- Frozen during diffusion training

The VAE from your earlier lectures!

Stage 2: Latent Diffusion

- U-Net with cross-attention
- Diffusion happens on z, not pixels
- Conditioning via cross-attention layers
- Text encoded by CLIP or T5

This is the architecture behind Stable Diffusion

Cross-Attention for Conditioning

How Text Guides Denoising

Q = projection of U-Net spatial features

K, V = projection of text encoder output (e.g., CLIP tokens)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Each spatial location attends to the relevant text tokens

What This Enables

"A cat sitting on a red sofa"

The cat region attends to "cat"

The sofa region attends to "red sofa"

Spatially-aware conditioning

Other Conditioning Modalities

ControlNet: depth, pose, edges

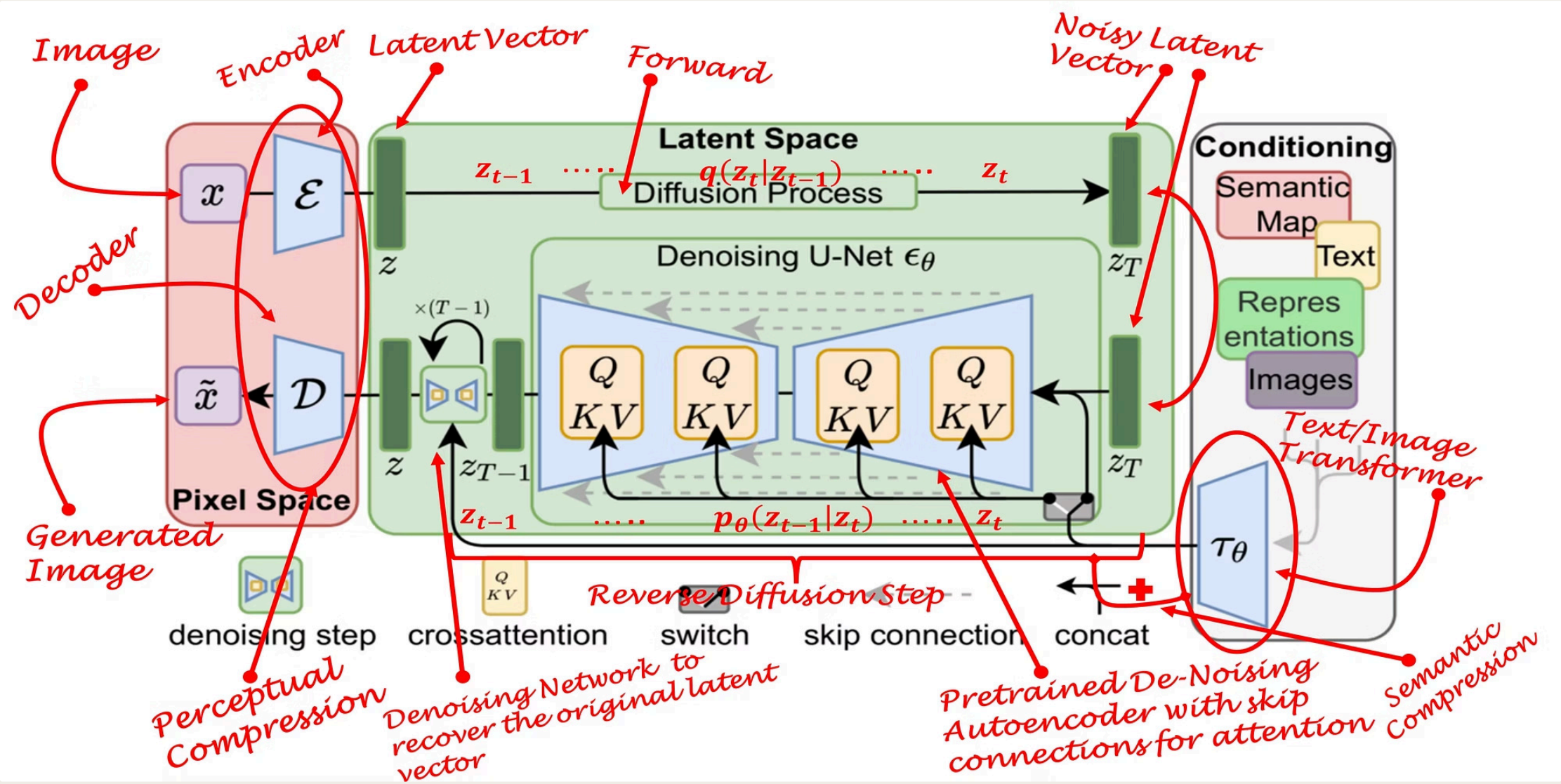
IP-Adapter: image prompts

Inpainting: masked regions

All injected via cross-attention or concatenation

Modular and extensible design

Latent Diffusion Models



07

Text-to-Image Models

The systems that brought diffusion to the mainstream

DALL-E 2

Key Paper: Ramesh et al. (2022) — OpenAI

Architecture

CLIP text encoder → prior network → CLIP image embedding → diffusion decoder

Two-stage: first generate a CLIP image embedding from text, then decode to pixels

Uses cascaded diffusion (64x64 → 256x256 → 1024x1024)

The prior is itself a diffusion model!

Key Innovation

CLIP as shared embedding space

Text and images mapped to same space

Capabilities

Text-to-image generation

Image variations and interpolation

Imagen

Key Paper: Saharia et al. (2022) — Google Brain

Architecture

T5-XXL text encoder → base diffusion (64x64) → super-res (256) → super-res (1024)

Pixel-space diffusion (no latent compression)

Cascaded super-resolution pipeline

Key finding: a larger text encoder matters more than a larger U-Net

Key Innovation

Text encoder quality is paramount

T5-XXL (11B params) vs CLIP (400M)

Dynamic thresholding for high CFG

Results

State-of-the-art FID on COCO

Excellent text-image alignment

Not publicly released (Google internal)

Stable Diffusion

Key Paper: Rombach et al. (2022) — CompVis / Stability AI / RunwayML

Architecture (LDM)

CLIP text encoder → KL-VAE encoder → U-Net (latent diffusion) → KL-VAE decoder

Diffusion in 64x64x4 latent space (for 512x512 images)

Text conditioning via cross-attention with CLIP embeddings

Single-stage (no cascaded super-res), much more efficient

Why It Changed Everything

Open-source weights and code

Runs on consumer GPUs (8GB VRAM)

Massive ecosystem: LoRA, ControlNet, ...

Evolution

SD 1.5 → SD 2.x → SDXL → SD 3.x

SD 3: DiT (transformer replaces U-Net)

FLUX, Playground, etc. — open ecosystem

Why Diffusion Won (Despite Being Slow)

Diffusion models are computationally expensive — so why did they become dominant?

The Cost

Training: expensive (many GPU-days)
Sampling: slow (hundreds of steps)

GANs: single forward pass at inference

The Payoff

Higher fidelity than GANs
Greater diversity (no mode collapse)

Much more stable training

Making It Practical: Acceleration Methods

Latent Diffusion — compress image first, diffuse in latent space (8-16x cheaper)

DDIM — skip steps during sampling (1000 → 50 steps with minimal quality loss)

Distillation — train a student model to do fewer steps (progressive distillation)

Modern diffusion can generate high-quality images in 1-4 steps — the speed gap is rapidly closing

08

Applications & Evaluation

What diffusion models can do and how we measure it

Evaluation Metrics

FID (Frechet Inception Distance)

Compares distributions of real and generated images in Inception feature space

Lower = better (0 = identical distributions)

Standard: 50k real vs 50k generated

Measures both quality and diversity

CLIP Score

Measures alignment between generated image and text prompt

Higher = better text-image correspondence

Doesn't measure image quality directly

Complementary to FID for text-to-image

Other Metrics & Considerations

- Inception Score (IS): quality + diversity, but less reliable than FID
- Precision/Recall: separates quality from diversity
- KID (Kernel Inception Distance): unbiased, works well with fewer samples
- Human evaluation still matters — metrics are imperfect proxies

What Evaluation Metrics Miss

No single metric captures what makes a generative model "good"

FID Limitations

- Does not measure prompt faithfulness
- Sensitive to sample size and preprocessing
- Can be gamed (memorization lowers FID)
- *Compares distributions, not individual images*

CLIP Score Limitations

- Does not measure image quality or realism
- CLIP has its own biases and blind spots
- High CLIP score is not always good image
- *Doesn't capture spatial or compositional errors*

What Really Matters

- Human evaluation remains the gold standard, but is expensive and subjective
- Benchmark scores depend strongly on setup, dataset, and protocol
- Different applications need different notions of quality (realism vs creativity vs controllability)
- *Always report multiple metrics and be transparent about limitations*

Critical thinking about evaluation is as important as understanding the models themselves

Applications

Image Generation

Text-to-image, class-conditional, unconditional synthesis

Image Editing

Inpainting, outpainting, SDEdit, instruction-based editing

Super-Resolution

Upscale low-res images with learned high-frequency detail

Video Generation

Temporal diffusion: Sora, Runway Gen-2, Stable Video

3D & Other Modalities

DreamFusion (3D), AudioLDM (audio), protein structure

Scientific Applications

Medical imaging, drug design, weather forecasting

09

Closing Synthesis

Diffusion in the history of generative AI

VAE vs GAN vs Diffusion

	VAE	GAN	Diffusion
Training	ELBO (stable)	Adversarial (unstable)	MSE (very stable)
Sample quality	Often blurry	Sharp, realistic	High quality + diverse
Mode coverage	Good coverage	Mode collapse risk	Excellent coverage
Speed (generate)	Fast (1 pass)	Fast (1 pass)	Slow (many steps)
Likelihood	Explicit (ELBO)	None (implicit)	Approximate
Controllability	Limited	cGAN, StyleGAN	CFG, ControlNet, etc.

Diffusion models combine the training stability of VAEs with image quality that matches or exceeds GANs

The Current Landscape

Diffusion Dominates

Text-to-image: Stable Diffusion, DALL-E 3, Midjourney, Imagen

Video: Sora, Runway, Stable Video

3D: DreamFusion, Zero-1-to-3

Audio: AudioLDM, MusicGen

GANs Still Matter

Real-time generation (StyleGAN)

Image editing and manipulation

Adversarial training in diffusion models

Distillation: SDXL Turbo uses adversarial loss

Emerging Trends

Flow matching (rectified flows)

Consistency models (single-step generation)

DiT: Transformers replace U-Nets

Distillation for faster inference

Open Questions

Can we generate in 1-2 steps reliably?

How to scale to longer video coherently?

What are the fundamental quality limits?

How to verify provenance of generated content?

Practical Tips for Diffusion

Training

- EMA (exponential moving average) on weights
- v-prediction or ε-prediction (both work)
- Cosine schedule often better than linear
- Drop condition 10% of time for CFG

Sampling

- DDIM for fast deterministic samples
- DPM-Solver++ for quality in ~20 steps
- CFG scale 5-7.5 for most use cases
- Negative prompts help avoid artifacts

If Training From Scratch

- Start small: 64x64 unconditional on a simple dataset (MNIST, CIFAR-10)
- Use AdamW, lr=1e-4 to 3e-4, batch size as large as GPU allows
- T=1000 for training, sample with DDIM 50-100 steps
- Visualize samples every N epochs — loss curves are less informative than in classification

Key Takeaways

01

Diffusion models generate by learning to reverse a noise-destruction process

02

Training is simple (MSE on noise prediction) and very stable — no adversarial games

03

Classifier-free guidance enables powerful conditional generation from text, images, etc.

04

Latent diffusion (LDM) made high-resolution generation practical on consumer hardware

05

The ecosystem is rich and modular: ControlNet, LoRA, IP-Adapter, and more

06

The main weakness is slow sampling, but DDIM, DPM-Solver, and distillation help

Essential Bibliography

Foundational

Ho et al. (2020), Denoising Diffusion Probabilistic Models — NeurIPS

Song et al. (2020), DDIM — ICLR 2021

Song et al. (2021), Score-Based Generative Modeling through SDEs — ICLR

Nichol & Dhariwal (2021), Improved DDPM — ICML

Text-to-Image Systems

Ramesh et al. (2022), DALL-E 2 — arXiv

Saharia et al. (2022), Imagen — NeurIPS

Rombach et al. (2022), LDM / Stable Diffusion — CVPR

Guidance & Conditioning

Dhariwal & Nichol (2021), Diffusion Models Beat GANs — NeurIPS

Ho & Salimans (2022), Classifier-Free Diffusion Guidance

Zhang et al. (2023), Adding Conditional Control (ControlNet) — ICCV

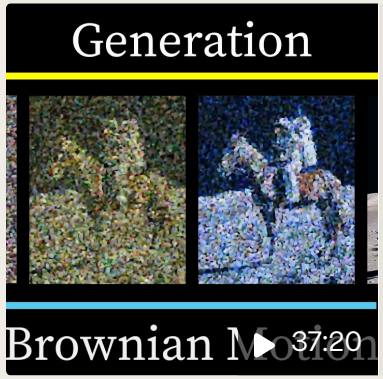
Modern Developments

Peebles & Xie (2023), DiT — ICCV

Song et al. (2023), Consistency Models

Lipman et al. (2023), Flow Matching — ICLR

Additional Resources



YouTube

But how do AI images and vide...

Diffusion models, CLIP, and the math of turning text into images Welch...

YouTube

How AI Image Generators Wor...

AI image generators are massive, but how are they creating such...